

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Name	Madhav.S	
Section	AIDS ‘B’	
Register Number	24011101066	

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

A single CNN architecture, MobileNetV2, was used throughout on the Oxford-IIIT Pet dataset. Each design choice was analyzed using training/validation curves, and a final configuration was selected using 5-fold cross-validation and evaluated once on an independent test set.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains 7,349 RGB images spanning 37 cat and dog breeds, loaded directly via `tensorflow_datasets`. Every image was resized to

$$224 \times 224 \times 3$$

and normalized using MobileNetV2’s own `preprocess_input` function, matching the ImageNet statistics the pre-trained base expects.

Split. The 7,349 images were stratified by breed into a 70/15/15 train/validation/test split:

Split	Number of Images
Training	5,144
Validation	1,102
Test	1,103

The test set was kept completely untouched until Section 8 (Final Model Evaluation) – every earlier section, including the 5-fold cross-validation in Section 7, used only the training split.

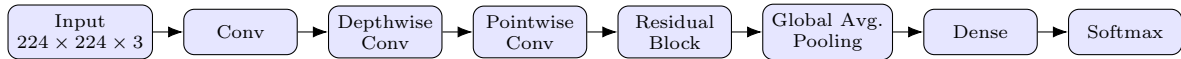
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

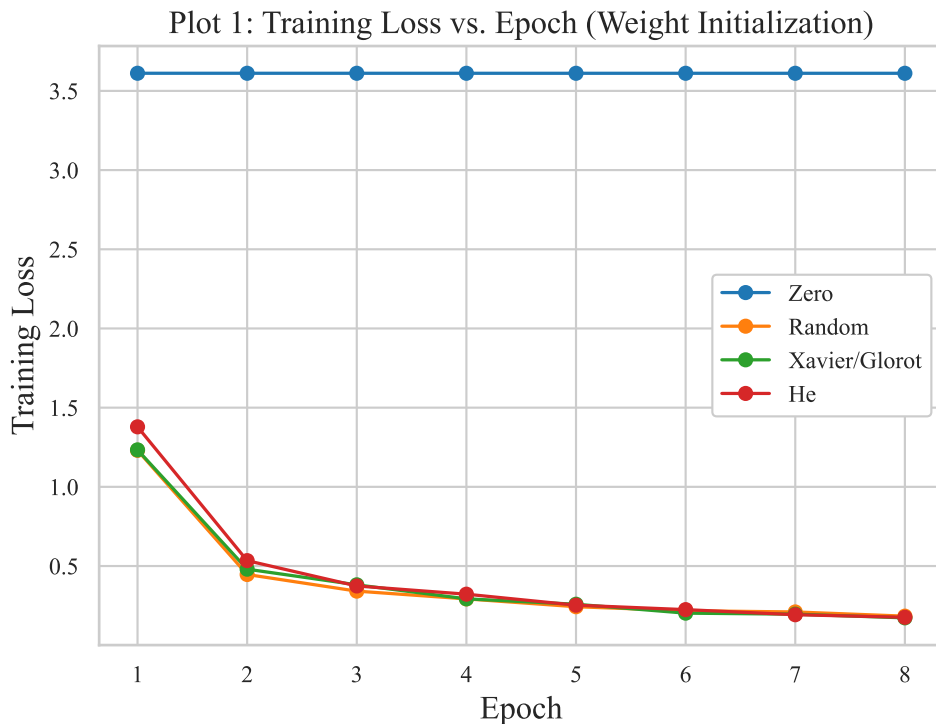
A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture. In this experiment, the pretrained ImageNet base is followed by GlobalAveragePooling2D, an optional BatchNormalization layer, a Dense(128, ReLU) layer, an optional Dropout layer, and a final Dense(37, softmax) layer.

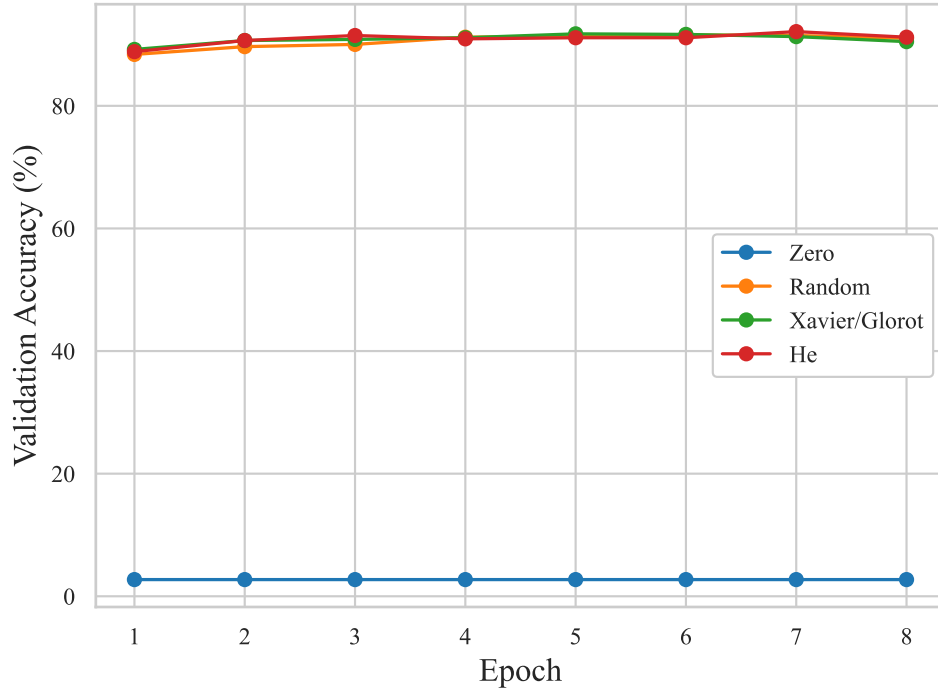
5. Weight Initialization

Four initialization strategies were compared for the newly-added classifier head (the pretrained convolutional base stayed frozen and retained its own ImageNet weights): Zero, Random (Normal), Xavier/Glorot and He, each trained for 8 epochs with Adam.



Inference: Zero initialization never leaves loss ≈ 3.61 because every head neuron receives an identical gradient and stays perfectly symmetric – the network cannot break symmetry on its own. Random, Xavier and He all fall from around 1.2–1.4 to under 0.2 within a couple of epochs, confirming that any asymmetric initialization is enough for this frozen-base setup to learn.

Plot 2: Validation Accuracy vs. Epoch (Weight Initialization)

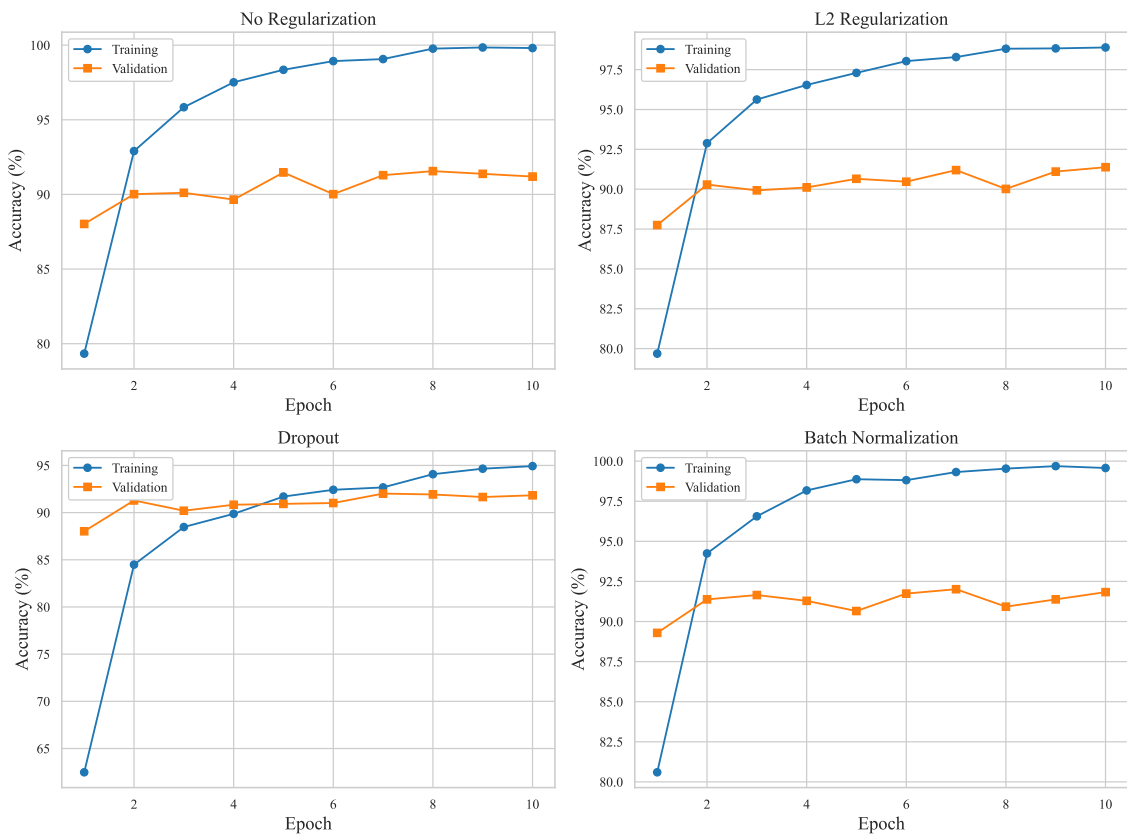


Inference: Zero initialization is stuck at 2.7% (random guessing among 37 classes), while Random, Xavier and He all climb past 90% validation accuracy. He initialization edges ahead slightly, peaking at 92.11%, with Xavier close behind at 91.74% – a small but expected gain, since He initialization is designed for the ReLU activation used in the classifier head.

6. Regularization and Overfitting

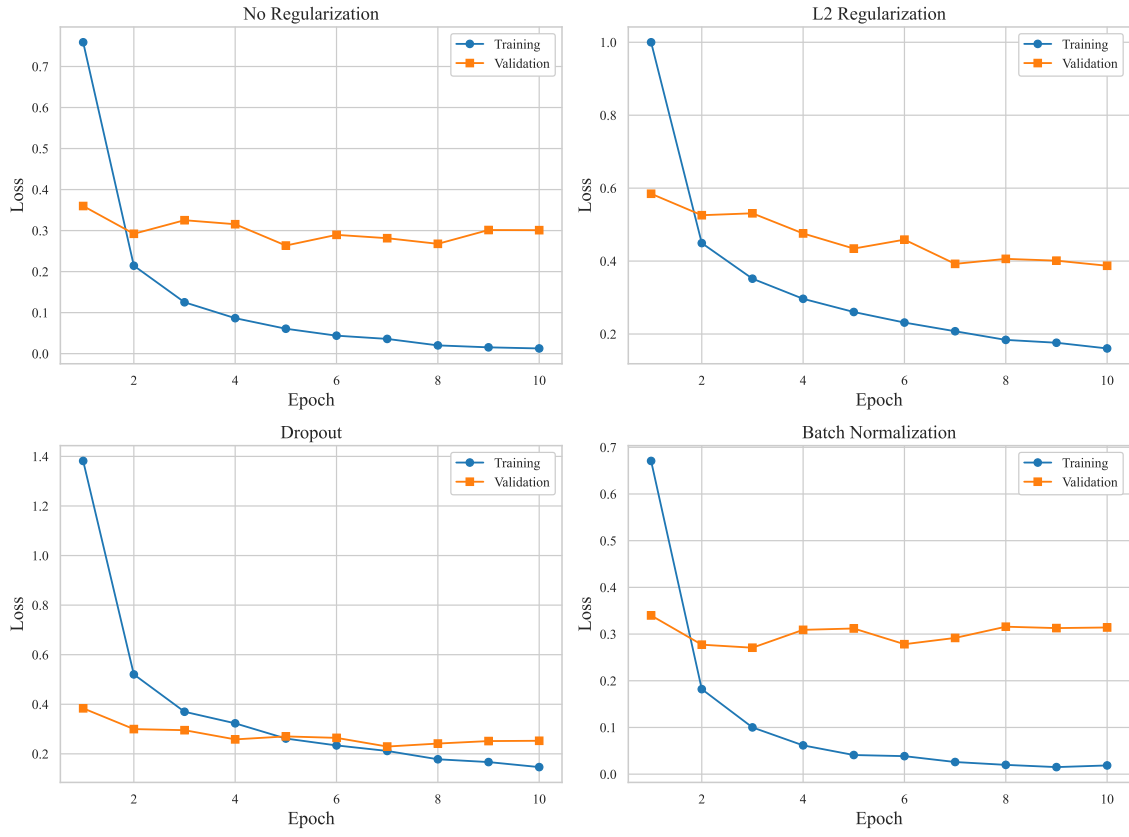
Four settings were compared for 10 epochs each: no regularization, L2 regularization ($\lambda = 10^{-3}$), Dropout (rate 0.5), and Batch Normalization on the head.

Plot 3: Training and Validation Accuracy vs. Epoch



Inference: With no regularization, training accuracy shoots past 99% by epoch 5 while validation accuracy plateaus around 91–92% – a clear, widening generalization gap. Dropout and L2 keep training accuracy lower for longer, and Batch Normalization converges fastest of all (training accuracy above 98% by epoch 4) .

Plot 4: Training and Validation Loss vs. Epoch



Inference: Training loss keeps falling in every configuration, but validation loss for “No Regularization” and “Batch Normalization” starts creeping back up after epoch 5–6 even as training loss keeps dropping. This shows overfitting. L2’s loss values sit visibly higher throughout, simply because the penalty term is added directly into the plotted loss, not because the model fits worse.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ ,

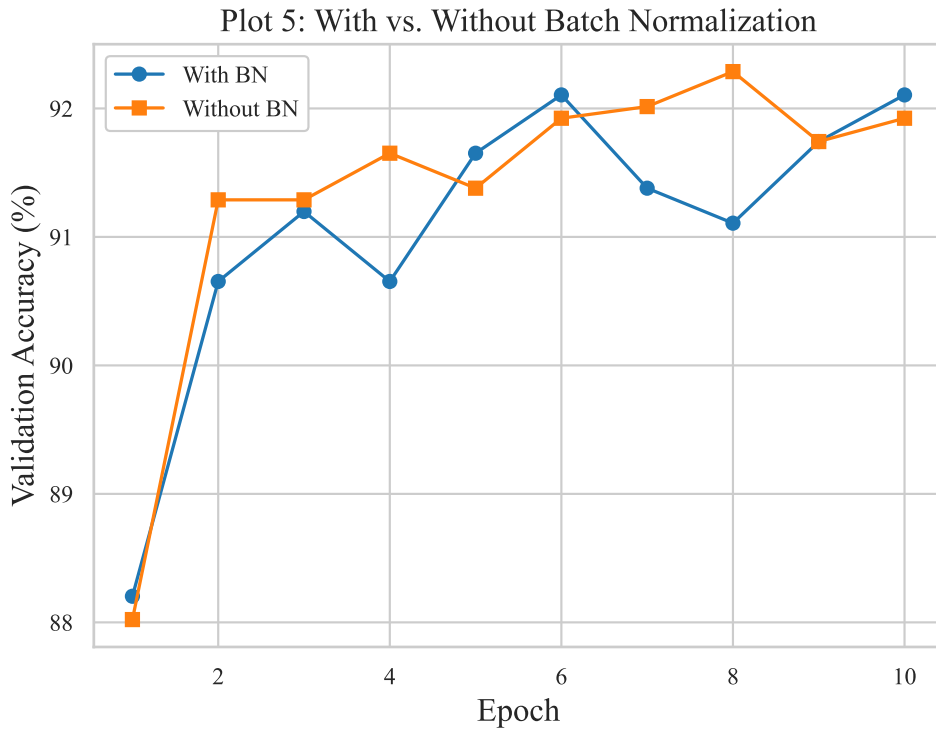
$$\begin{aligned} \hat{x}_1 &= \frac{2-5}{2.236} \approx -1.342, & \hat{x}_2 &= \frac{4-5}{2.236} \approx -0.447, \\ \hat{x}_3 &= \frac{6-5}{2.236} \approx 0.447, & \hat{x}_4 &= \frac{8-5}{2.236} \approx 1.342. \end{aligned}$$

Hence,

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

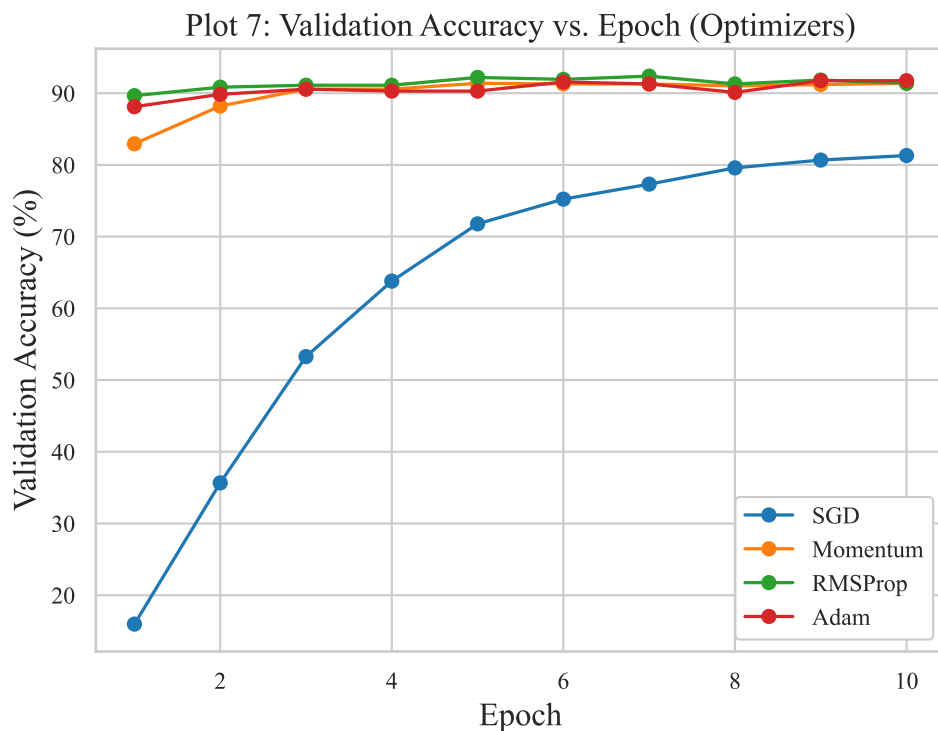
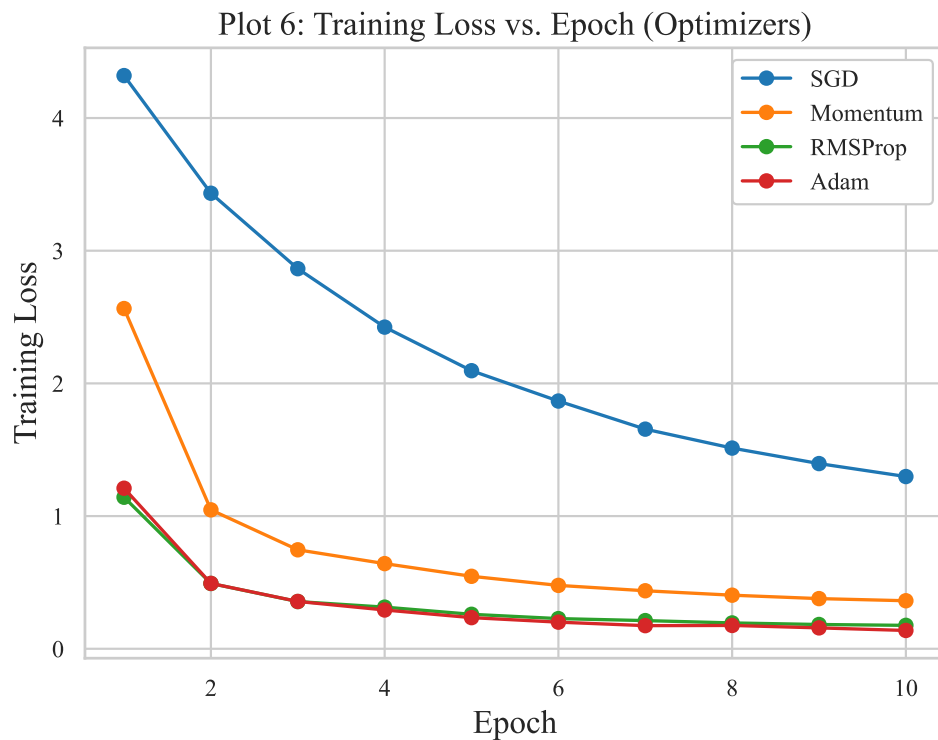
Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift instead of being forced to keep every activation standardized.



Inference: With BN peaks at 92.01% (epoch 7) and Without BN peaks slightly higher at 92.29% (epoch 8) – BN sped up training accuracy noticeably (past 98% by epoch 4 versus a slower climb without it) but did not translate into a clearly better validation accuracy here, most likely because MobileNetV2’s frozen convolutional base is already full of its own internal BatchNorm layers, so adding one more on top of an already-normalized feature vector has limited extra benefit.

8. Optimization Algorithms

SGD, Momentum, RMSProp and Adam were compared for 10 epochs each, all other settings held fixed (Dropout 0.5, BatchNorm on, learning rate 10^{-3}).



Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time
SGD	1.2980	81.31%	10	74.6 s
Momentum	0.3619	91.38%	5	74.9 s
RMSProp	0.1765	92.38%	7	74.7 s
Adam	0.1379	91.74%	9	75.0 s

Inference: Plain SGD is still climbing at epoch 10 and lands over 10 points behind every adaptive method – with no momentum term, a 10^{-3} learning rate is simply too small to move quickly through this loss surface. RMSProp reaches the best validation accuracy (92.38%) fastest of the three adaptive methods, while Adam achieves the lowest final training loss; both clearly outperform Momentum and SGD within the same 10-epoch, 75-second training budget.

9. CNN Hyperparameter Tuning

Learning rate, batch size and dropout rate were each swept independently (6 epochs, one hyperparameter changed at a time, all others fixed at LR = 10^{-3} , batch size 32, dropout 0.25):

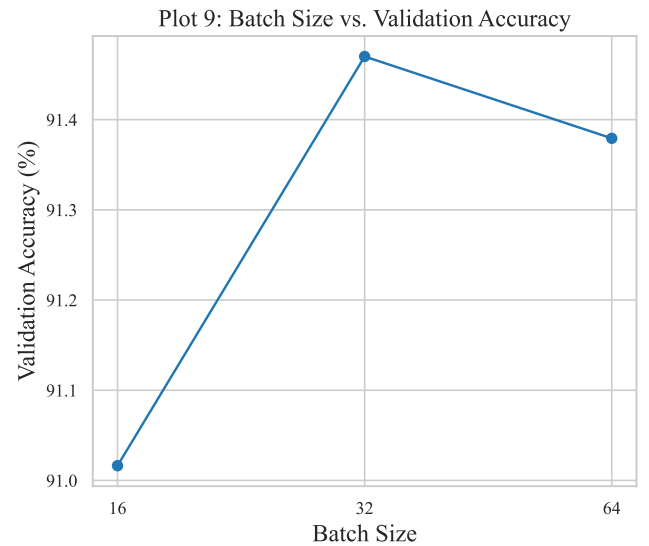
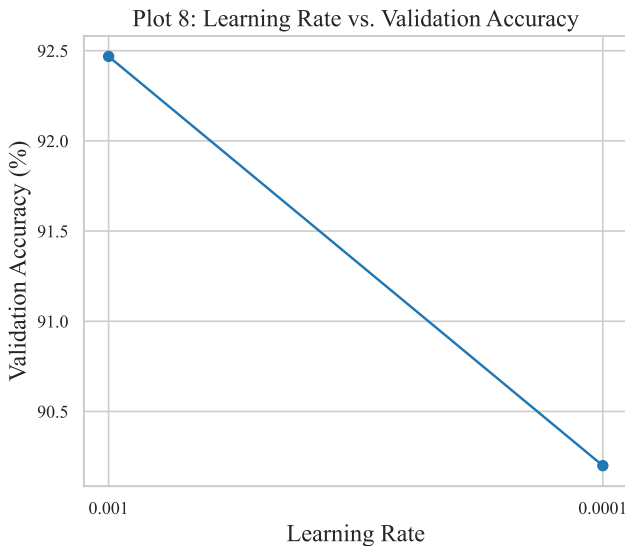
Hyperparameter	Values Tested	Result (Best Val. Accuracy)
Learning Rate	0.001, 0.0001	0.001 $\rightarrow$ 92.47%; 0.0001 $\rightarrow$ 90.20%
Batch Size	16, 32, 64	16 $\rightarrow$ 91.02%; 32 $\rightarrow$ 91.47%; 64 $\rightarrow$ 91.38%
Dropout Rate	0, 0.25, 0.5	0 $\rightarrow$ 91.38%; 0.25 $\rightarrow$ 91.56%; 0.5 $\rightarrow$ 91.83%

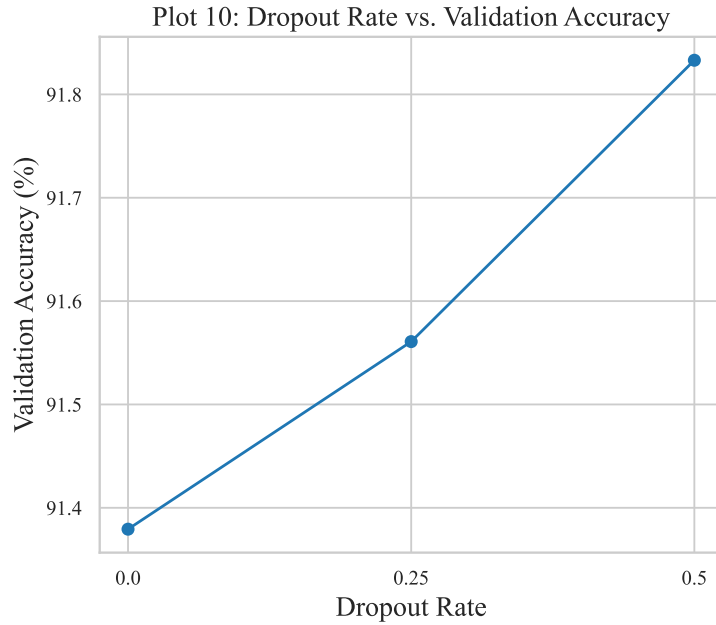
Optimizer choice (Section 8) and the frozen-base vs. fine-tuning trade-off (Section 10) were investigated separately, since both already required their own dedicated comparisons above.

For CNN concepts, the following were also reviewed: kernel/filter, stride, padding, pooling, number of channels, and depthwise separable convolution. For a convolution operation, the output dimension is

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride. For example, $N=224, K=3, P=1, S=1$ gives $O = 224$ (same-padding, size-preserving), while $N=224, K=3, P=0, S=2$ gives $O = 111$ (a strided, downsampling convolution).





Inference: The higher learning rate (10^{-3}) beats the lower one by over 2 points within this short 6-epoch – 10^{-4} simply hasn't taken enough steps yet. Batch size 32 is a mild sweet spot between 16 and 64, and accuracy rises steadily with more dropout (0 to 0.5), suggesting the classifier head is prone to overfitting quickly and benefits from stronger regularization even in this short run.

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet was used for both cases below.

Case A: Feature Extraction

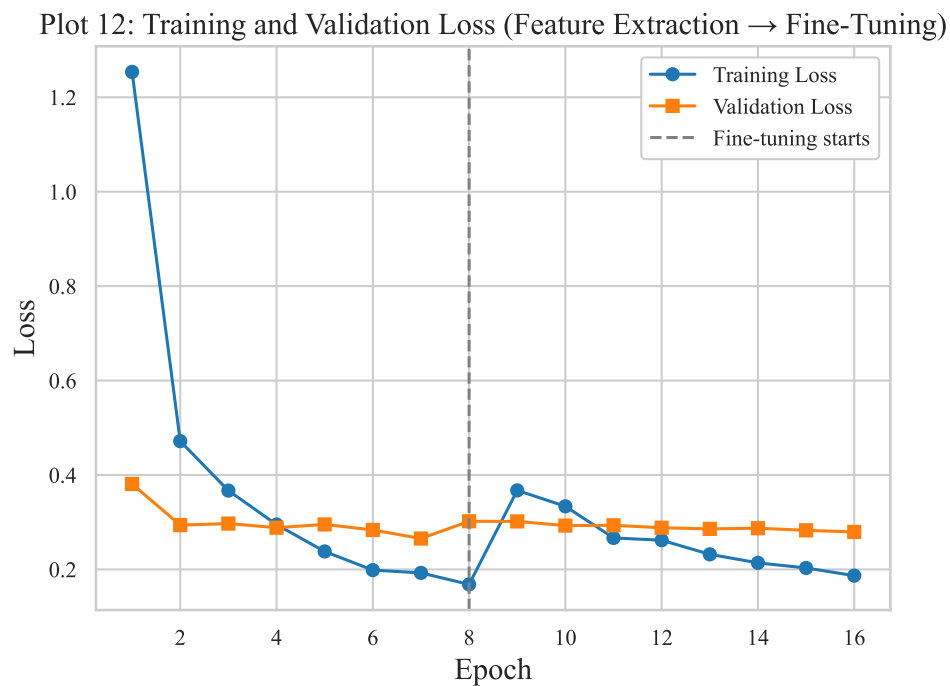
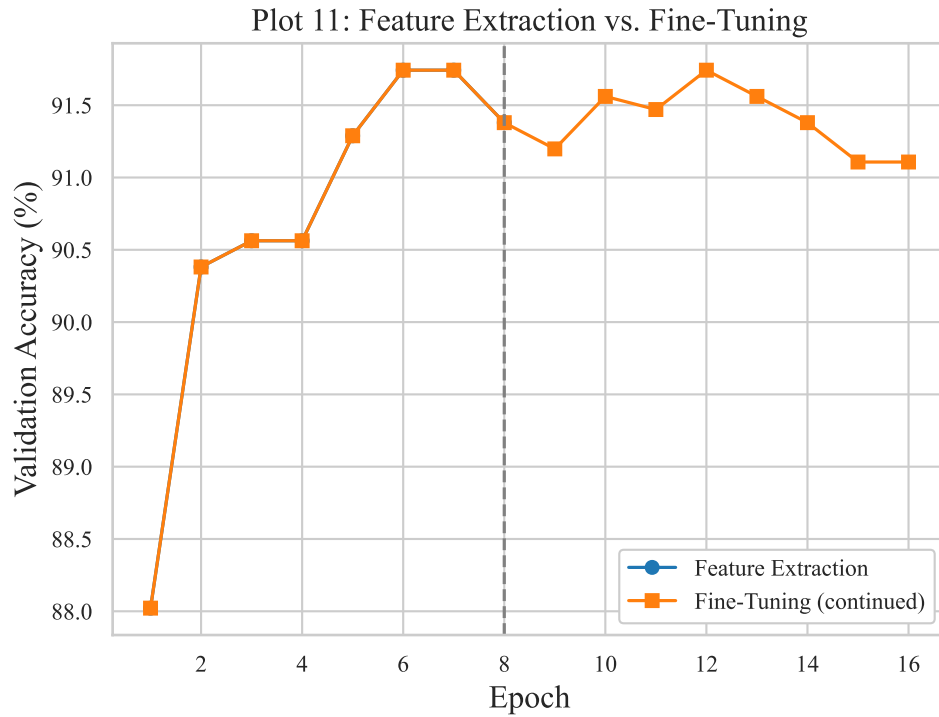
Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers were frozen and only the newly added classification head was trained for 8 epochs.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Last 30 Layers → LR = 10^{-5}

The last 30 layers of the pretrained base were unfrozen and trained for a further 8 epochs at a much smaller learning rate (10^{-5} vs. 10^{-3}), continuing from the feature-extraction weights.



Inference: Feature extraction alone already reaches 91.74% validation accuracy by epoch 7, and fine-tuning the last 30 layers does not push past that peak within its own 8 epochs (settling around 91.1–91.7%). With a frozen-base model already this strong, a longer fine-tuning run or unfreezing more layers would likely be needed to see a clearer gain.

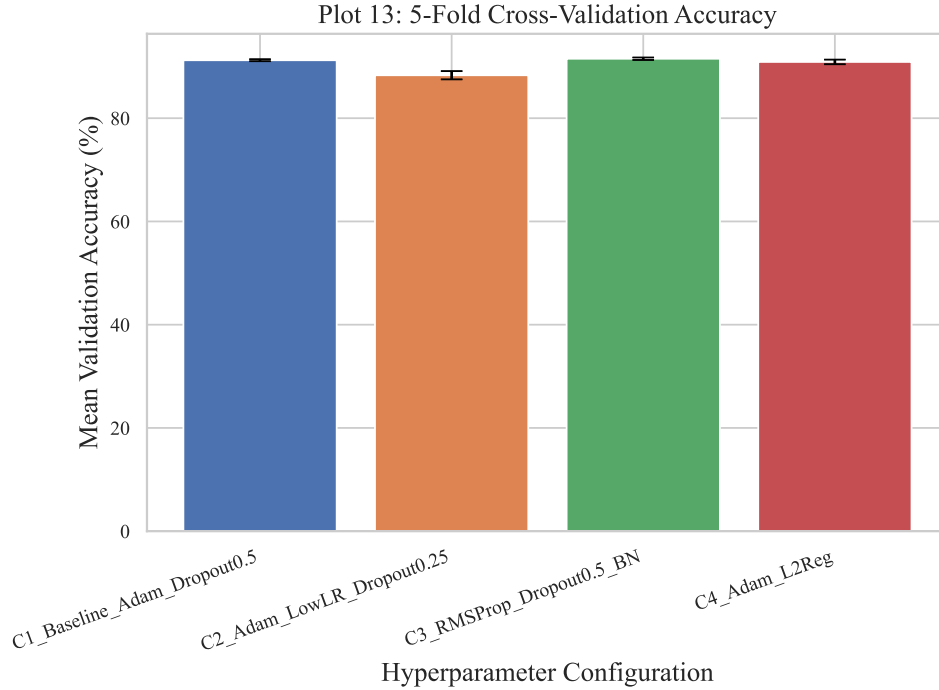
Discussion: Fine-tuning uses a smaller learning rate than training a freshly-initialized classifier because the unfrozen layers already hold well-trained ImageNet weights; a large learning rate would overwrite those useful features with large, disruptive updates before the new classifier has had a chance to guide them gently.

11. K-Fold Cross-Validation

Four configurations, chosen based on the results of the sections above, were evaluated using 5-fold cross-validation on the training split only (5 epochs per fold):

Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD
C1: Adam, Dropout 0.5, BN	91.06	91.35	91.35	91.45	90.95	91.23 ± 0.19
C2: Adam (LR 10^{-4}), Dropout 0.25, BN	88.14	87.56	88.82	87.46	89.59	88.32 ± 0.80
C3: RMSProp, Dropout 0.5, BN	91.06	91.64	91.74	91.55	91.63	91.52 ± 0.24
C4: Adam, L2 = 10^{-3} , BN	90.77	90.96	91.74	90.48	90.56	90.90 ± 0.45

$$\bar{A}_{C3} = 91.52\% \pm 0.24$$

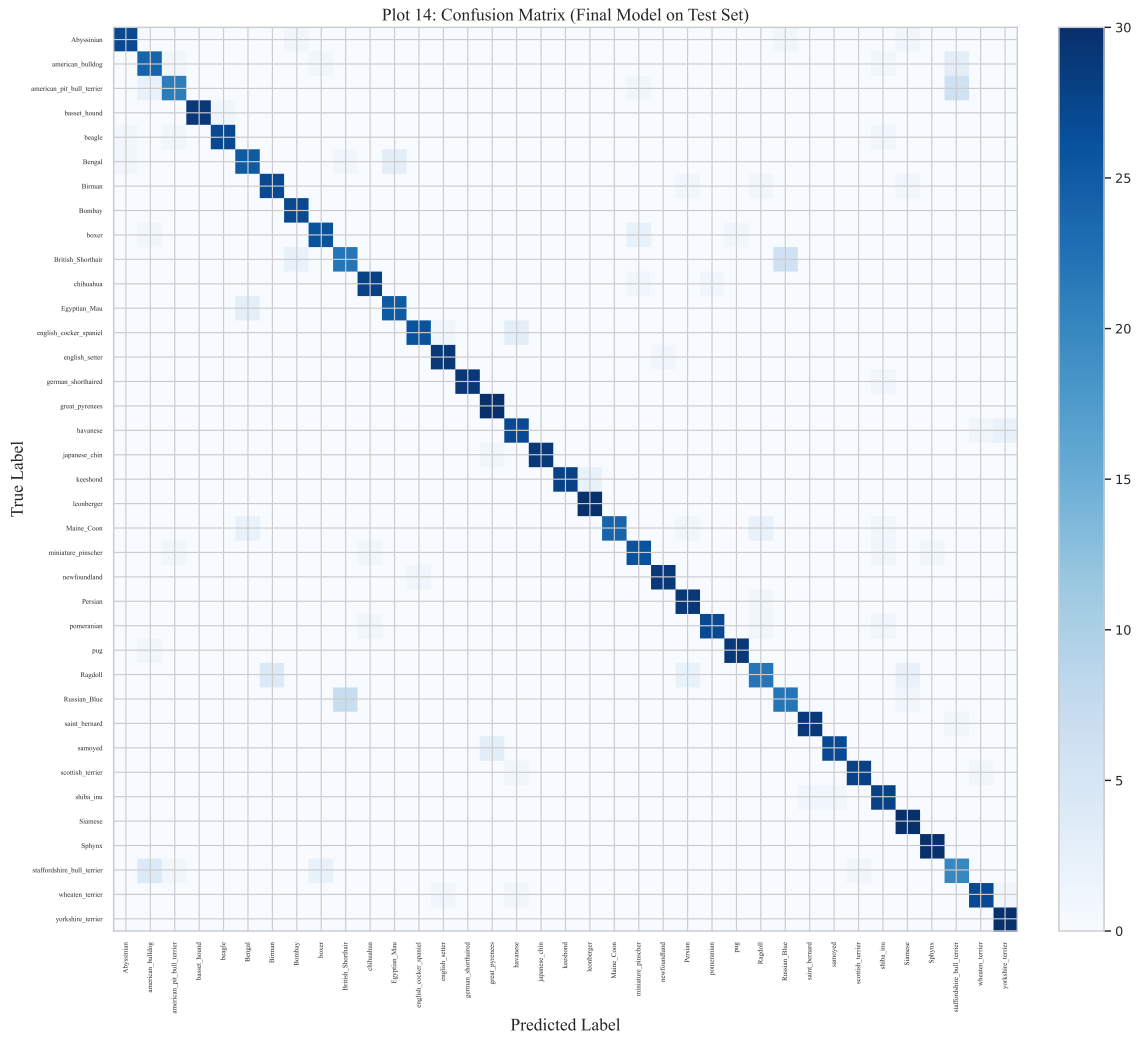


Inference: C3 (RMSProp, Dropout 0.5, BN) wins on both counts that matter – the highest mean accuracy *and* the joint-lowest standard deviation (tied with C1) – so it was selected as the final configuration. C2’s low learning rate (10^{-4}) clearly needed more than 5 epochs per fold to catch up, both scoring lowest and varying most (SD = 0.80) across folds.

12. Final Model Evaluation

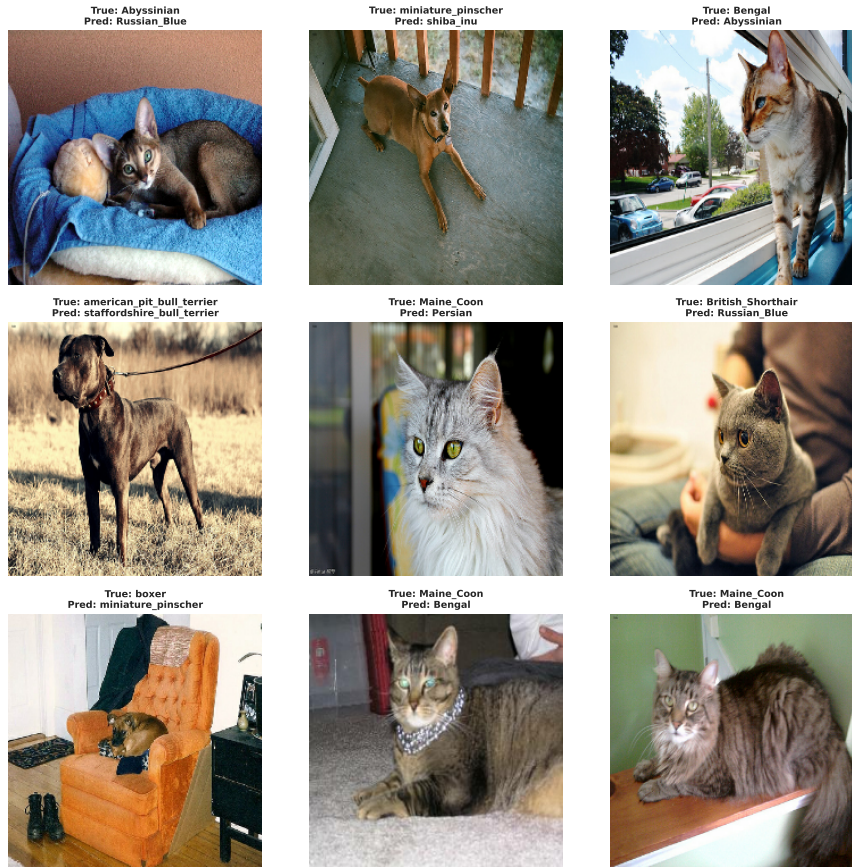
Configuration C3 (RMSProp, Dropout 0.5, Batch Normalization) was retrained on the complete training split for 12 epochs and evaluated once on the untouched test set:

Metric	Value
Mean CV Accuracy	91.52%
CV Standard Deviation	0.24
Test Accuracy	90.03%
Precision (macro)	0.9021
Recall (macro)	0.9002
F1-score (macro)	0.8997
Training Time	87.1 s
Number of Parameters	2,431,845



Inference: Several breeds are classified with perfect accuracy on the test set (Yorkshire Terrier, Siamese, Sphynx, Great Pyrenees, Leonberger), while the confusion that does occur is concentrated in visually similar breed pairs – Russian Blue vs. British Shorthair (grey short-haired cats) and American Pit Bull Terrier vs. Staffordshire Bull Terrier (near-identical short-haired dog breeds) are the two most frequently confused pairs.

Plot 15 (Optional): Representative Misclassified Images



Inference: The sampled misclassified images mostly show partial views, unusual poses or lighting, or a genuinely similar-looking breed being predicted – consistent with the confusion pairs identified above rather than any obvious data or labelling problem.

13. Overall Results

Configuration	CV Accuracy	SD	Test/Val. Accuracy	Training Time
Baseline (No Regularization)	–	–	91.56% (val)	–
Best Initialization (He)	–	–	92.11% (val)	–
Best Regularization (Dropout, tied with BN)	–	–	92.01% (val)	–
Best Optimizer (RMSProp)	–	–	92.38% (val)	74.7 s
Best Hyperparameters (LR = 10^{-3})	–	–	92.47% (val)	–
Fine-Tuned Model	–	–	91.74% (val)	–
Final Selected Model (C3)	91.52%	0.24	90.03% (test)	87.1 s

The individual studies (initialization, regularization, optimizer, hyperparameters, fine-tuning) each report the best *validation* accuracy from their own single run, since only the final, cross-validated configuration was retrained and evaluated on the held-out test set. The final model's test accuracy is 90.03%. C3 was selected over the individually-higher-scoring ablation runs above because it was the only configuration validated for both accuracy *and* consistency (low standard deviation) across five independent folds, rather than a single train/validation split.

14. Discussion Questions

- What is the difference between model parameters and hyperparameters?** Parameters (weights, biases) are learned automatically during training via backpropagation. Hyperparameters (learning rate, batch size, dropout rate, number of layers) are set by the practitioner before training and control how that learning happens.
- Why is weight initialization important?** It determines the starting point of optimization. A poor starting point can cause vanishing/exploding activations or, in the worst case (zero initialization), prevent the network from learning at all.

3. **Why can zero initialization be problematic for neural networks?** Every neuron in a layer receives the same input and the same gradient, so they all update identically forever – the network stays perfectly symmetric and effectively behaves like a single neuron per layer, as seen directly in Section 5 (stuck at 2.7% accuracy).
4. **Compare Xavier and He initialization.** Both keep the variance of activations stable across layers, but Xavier assumes a linear/tanh-like activation while He accounts for ReLU zeroing out roughly half the activations, using a variance scaled by $2/n$ instead of $1/n$. He is generally preferred for ReLU networks, matching its slightly better result here (92.11% vs. 91.74%).
5. **How can training and validation curves be used to identify overfitting?** Overfitting is visible when training accuracy keeps rising (or training loss keeps falling) while validation accuracy plateaus or validation loss starts rising – exactly what appeared for the “No Regularization” configuration in Section 6.
6. **How does Dropout reduce overfitting?** By randomly zeroing a fraction of neurons on each training step, it prevents co-adaptation between neurons and forces the network to learn more redundant, robust features rather than memorizing the training set.
7. **What is the purpose of Batch Normalization?** It normalizes layer activations within each mini-batch, stabilizing and speeding up training, reducing sensitivity to initialization, and providing a mild regularizing effect.
8. **Explain the numerical Batch Normalization example.** For $x = [2, 4, 6, 8]$, the mean is 5 and variance is 5, giving normalized values $\approx [-1.342, -0.447, 0.447, 1.342]$ – centred at zero with unit variance, exactly as Section 7 works out step by step.
9. **What are the roles of γ and β ?** They let the network undo the normalization if that is what’s optimal – γ rescales and β re-shifts the normalized activations, so BatchNorm never strictly limits what the layer can represent.
10. **Compare SGD, Momentum, RMSProp and Adam.** SGD is the simplest and slowest to converge, as seen in Section 8 (still at 81.31% after 10 epochs). Momentum accelerates it using past gradients. RMSProp and Adam both use per-parameter adaptive learning rates, converging fastest and most reliably here (92.38% and 91.74% respectively).
11. **What happens when the learning rate is too large?** Updates overshoot the minimum, and loss can oscillate or diverge instead of decreasing steadily.
12. **What happens when the learning rate is too small?** Training becomes very slow to converge, as seen with $LR = 10^{-4}$ in Section 9, which still trailed the 10^{-3} run by over 2 points after the same number of epochs.
13. **What is the effect of increasing batch size?** Gradient estimates become less noisy (more stable updates) but there are fewer updates per epoch; very large batches can also generalize slightly worse, matching batch size 64 scoring marginally below 32 in Section 9.
14. **Explain stride and padding.** Stride is the step size the convolution kernel moves across the input; larger strides downsample the output. Padding adds extra pixels (typically zeros) around the input border, letting the output size be controlled independently of the kernel size (e.g. “same” padding keeps the spatial size unchanged).
15. **Why is MobileNetV2 computationally efficient?** It replaces standard convolutions with depthwise separable convolutions (a depthwise pass followed by a 1×1 pointwise pass), which need far fewer multiply-accumulate operations for a similar receptive field.
16. **What is depthwise separable convolution?** A standard convolution factored into two cheaper steps: a depthwise convolution that filters each input channel independently, followed by a 1×1 pointwise convolution that combines the channels.
17. **What is transfer learning?** Reusing a model already trained on a large dataset (ImageNet) as the starting point for a new, smaller task – implemented here for the Oxford-IIIT Pet dataset via MobileNetV2.
18. **Differentiate feature extraction and fine-tuning.** Feature extraction keeps the entire pretrained base frozen and only trains the new classifier head. Fine-tuning additionally unfreezes some of the base’s later layers and trains them (at a small learning rate) so they adapt to the new dataset.
19. **Why is a smaller learning rate generally used during fine-tuning?** The unfrozen layers already hold useful pretrained weights; a large learning rate would apply large, disruptive updates that overwrite those features before the classifier can guide them usefully.
20. **Why is K-Fold Cross-Validation useful for hyperparameter selection?** It evaluates a configuration across several different train/validation splits instead of just one, giving both a more reliable accuracy estimate and a measure of how much that estimate varies (Section 11).
21. **Why must the test set remain untouched during tuning?** If the test set influences any decision (choice of hyperparameters, architecture, or model), it stops being an independent measure of generalization and the reported accuracy becomes optimistically biased.
22. **Why should mean and standard deviation both be reported?** Mean accuracy alone hides how consistent a configuration is; two configurations with the same mean can have very different reliability, as seen between C1 (SD 0.19) and C4 (SD 0.45) in Section 11.
23. **Is the highest validation accuracy always sufficient to select a model?** No – a single high validation score can come from a lucky split. C3 was chosen here not for having the single highest number anywhere in the notebook, but for combining a high *mean* with a low standard deviation across five independent folds.

15. Additional Exercise: New Configurations via 5-Fold Cross-Validation

Two new combinations of learning rate, dropout, batch size and fine-tuning strategy were evaluated with 5-fold cross-validation and compared against the previously selected configuration, C3:

Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD
N1: LR 10^{-4} , Dropout 0.5, Batch 16, Fine-Tuned	89.60	90.96	89.70	87.95	89.40	89.52 ± 0.96
N2: LR 10^{-3} , Dropout 0.1, Batch 64, Feature-Extraction	89.70	91.45	91.55	89.50	90.76	90.59 ± 0.85
C3 (previous best, for comparison)	91.06	91.64	91.74	91.55	91.63	91.52 ± 0.24

Inference: Neither new configuration beats C3 on mean accuracy or consistency. N1 combines a low learning rate with fine-tuning, but its short 3-epoch fine-tuning phase and small batch size leave it both the lowest-scoring and the most variable (SD = 0.96) of the three. N2 does better without any fine-tuning at all, suggesting the higher learning rate and larger batch size are more responsible for its improvement over N1 than the frozen-vs-fine-tuned choice is.

16. Conclusion

- Compared four weight initialization schemes on a frozen MobileNetV2 classifier head: Zero initialization failed completely (2.7%, random-guess level), while Random, Xavier and He all converged normally, with He slightly ahead at 92.11%.
- Demonstrated overfitting directly: an unregularized head reached 99.8% training accuracy while validation accuracy plateaued near 91–92%; Dropout and Batch Normalization both curbed this gap.
- Compared SGD, Momentum, RMSProp and Adam under identical conditions – RMSProp reached the best validation accuracy (92.38%), with plain SGD trailing by over 10 points in the same 10-epoch budget.
- Swept learning rate, batch size and dropout rate independently, finding LR = 10^{-3} , batch size 32, and dropout 0.5 each performed best within their own sweep.
- Implemented both feature extraction and fine-tuning on MobileNetV2; fine-tuning the last 30 layers did not clearly improve on feature extraction’s 91.74% peak within an 8-epoch budget.
- Selected a final configuration (RMSProp, Dropout 0.5, Batch Normalization) using 5-fold cross-validation on four candidates ($91.52\% \pm 0.24$, the best mean *and* joint-lowest variability), then evaluated it once on a completely untouched test set, reaching 90.03% test accuracy with a macro F1-score of 0.8997.
- Two additional hyperparameter combinations were tested via 5-fold CV and both fell short of the selected configuration, confirming it as the preferred choice.

17. Source Code

The complete implementation, exactly as run on Kaggle, is hosted on GitHub.

https://github.com/smadhav2024/Deep_Learning_Lab/tree/main/Lab%205

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, “Cats and Dogs,” IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>